Verasonics®
The leader in Research Ultrasound™

# Vantage® *NXT* Research Ultrasound Systems

## FastSave Tutorial: L22-14vX FlashAngles

Verasonics Inc. | 11335 NE 122nd Way, Ste 100, Kirkland, WA 98034 | www.verasonics.com
©Verasonics Inc. 2001- 2025. Verasonics Confidential.

## Table of Contents

*1. Tutorial Overview* ..... *2*

*2. Profile the existing script* ..... *2*

*3. Hardware frame timing and constraints* ..... *2*

*4. Choosing the number of sub-frames per super-frame* ..... *2*

*5. Modify the sequence script (high-level steps)* ..... *3*

*6. Test and validate* ..... *7*

*7. Example summary statistics* ..... *8*

*8. Recommendations and notes* ..... *8*

# 1. Tutorial Overview

This tutorial shows how to modify a L22-14vX FlashAngles sequence to use FastSave for continuous RF streaming while retaining live monitoring with reduced reconstruction rates.

# 2. Profile the existing script

```
matlab

    SetUpL22_14vXFlashAngles
    vsProfile start
    VSX
```

Run the script for ~60sec to collect enough statistics, then stop and exit the sequence.

```
vsProfile stop[resultTable, results, benchmarks] = vsProfile( 'review');

>> results(2).MedianDuration

ans =

24.4675
```

This result says that it takes about 24.5ms to reconstruct 1 frame of 7 plane waves compounded.

```
>> Receive(1).endSample*na*128*2

ans =

 4358144
```

- Median reconstruction time: ~24.5 ms per compounded frame (7 angles).
- DMA size per sub-frame: 4,358,144 bytes (≈ 4.16 MiB / 4.36 MB). This relatively small transfer size will not fully saturate the DMA capabilities of the hardware; grouping sub-frames into larger super-frames is recommended.

# 3. Hardware frame timing and constraints

timeToNextAcq (between angle acquisitions) = 160us

timeToNextAcq (between frames) = 19040us

This original time between frames is set for a hardware framerate of 50fps. This can be reduced to a minimum value of the time between angle acquisitions.

- Therefore, one compounded frame (7 angles) duration: 7 × 160 µs = 1,120 µs ≈ 1.12 ms, i.e., ~893 frames/s

**Note**: Reconstruction (≈24.5 ms) is much slower than the raw hardware acquisition rate. To provide live monitoring without dropping frames, we must group multiple sub-frames into a super-frame for DMA transfer and perform reconstruction at a lower effective frame rate.

# 4. Choosing the number of sub-frames per super-frame

Constraints and calculations:

- Reconstruction time must be less than the super-frame acquisition time. With a reconstruction time of 24.5 ms and a single-frame acquisition time of ~1.12 ms:Number of sub-frames >= 24.5 ms / 1.12 ms ≈ 21.9 ⇒ choose ≥ 23 sub-frames.
    - Super-frame acquisition time (23 sub-frames) = 1.12 ms × 23 ≈ 25.8 ms → ≈ 38.8 super-frames/s.
- DMA transfer efficiency: empirically, larger DMA transfers (≥ 100 MB) achieve higher sustained DMA rates on this platform. Using the per-sub-frame sizeExample (25 sub-frames): 25 × 4,358,144 = 108,953,600 bytes ≈ 103.9 MiB (108.95 MB).
    - Per sub-frame: 4,358,144 bytes (~4.16 MiB / 4.36 MB).
    - Minimum sub-frames for ~100 MB: 100,000,000 / 4,358,144 ≈ 22.9 ⇒ choose ≥ 23 (rounded up to an integer).
- Upper bound from per-file limits: ensure the super-frame size remains below any 32-bit/implementation limits (e.g., 2 GB):Max sub-frames (≈2,000 MB / 4.36 MB) ≈ 458 ⇒ safely above the operating range.

Recommendation for this example: choose 50 sub-frames for higher DMA efficiency while maintaining a usable live reconstruction rate.

- Super-frame DMA size (50 sub-frames): 50 × 4,358,144 = 217,907,200 bytes ≈ 207.7 MiB (217.9 MB).
- Super-frame acquisition time: 1.12 ms × 50 ≈ 56 ms ⇒ effective live reconstruction rate ≈ 17.9 fps (≈18 fps).
- Minimum DMA transfer duration (approx): 0.218 GB / 7 GB/s ≈ 31 ms (order-of-magnitude estimate); this is consistent with the observed acquisition timing for a sustained transfer.

These calculations show that 25 sub-frames is close to the minimum acceptable value; 50 sub-frames is a reasonable compromise for higher DMA efficiency and an ~18 fps live reconstruction rate.

## 5. Modify the sequence script (high-level steps)

Reduce the inter-frame wait (set timeToNextAcq between frames to the per-angle interval so frames are captured continuously). For example, update the SeqControl argument used to set the frame interval (replace the previous 20 ms-derived argument with 160):
Below are the minimal, focused edits to adapt the example sequence. Preserve existing sequence semantics where possible and test incrementally.

```
matlab

%% before
SeqControl(3).argument = 20000 - (na-1)*160;  % example prior value (20 ms)

%% after (set inter-angle / inter-frame spacing to 160 us)
SeqControl(3).argument = 160;  % microseconds
```

Set the chosen number of sub-frames per super-frame (example uses 50):

```
matlab

P.nSubFrames = 50;
```

Configure FastSave variables (base filename + target drives):

```
matlab

fsFileName = 'testFileName';
if isunix()
    saveMethod = 'fastsaveLinux';
    fsDriveNames =
{'/media/verasonics/WD1','/media/verasonics/WD2','/media/verasonics/WD3','/media/ve
rasonics/WD4'};
elseif ispc()
    saveMethod = 'fastsaveWindows';
    fsDriveNames = {'D:\','E:\','F:\','G:\'};
end
```

Create the FastSave external process entry:

```
matlab

fastSaveProcess = 2; % choose an available process index
Process(fastSaveProcess).classname = 'External';
Process(fastSaveProcess).method = saveMethod;
Process(fastSaveProcess).Parameters = { ...
    'srcbuffer','receive', ...
    'srcbufnum', 1, ...
    'srcframenum', -1, ... % process the last frame in the receive buffer
    'dstbuffer','none' ...
};
```

Add external save events and required SeqControls to transfer and mark DMA transfers:

```
matlab

% Example snippet (integrate into the Event creation loop):
Event(n).info = 'Transfer To Host';
Event(n).tx = 0; Event(n).rcv = 0; Event(n).recon = 0; Event(n).process = 0;
Event(n).seqControl = [nsc, nsc + 1];
SeqControl(nsc).command = 'transferToHost';
SeqControl(nsc+1).command = 'waitForTransferComplete';
SeqControl(nsc+1).argument = nsc; % wait index
nsc = nsc + 2;
n = n + 1;

% Save-event marker for grouping sub-frames (markTransferProcessed)
saveEvents = [saveEvents n];
Event(n).info = 'save all subframes';
Event(n).tx = 0; Event(n).rcv = 0; Event(n).recon = 0; Event(n).process = 0;
Event(n).seqControl = nsc;
SeqControl(nsc).command = 'markTransferProcessed';
SeqControl(nsc).argument = nsc - 2;
nsc = nsc + 1;
n = n + 1;
```

Create the super-frame receive structure and populate per-subframe attributes. The original structure is preserved; ensure `Receive` is sized for `na * P.nSubFrames * Resource.RcvBuffer(1).numFrames` entries and that `framenum`/`acqNum` are assigned consistently. Example:

```
matlab

maxAcqLength = ceil(sqrt(P.endDepth^2 + ((Trans.numelements-1)*Trans.spacing)^2));
Receive = repmat(struct('Apod', ones(1,Trans.numelements), ...
    'startDepth', P.startDepth, ...
    'endDepth', maxAcqLength, ...
    'TGC', 1, 'bufnum', 1, 'framenum', 1, 'acqNum', 1, ...
    'sampleMode', 'NS200BW', 'mode', 0, 'demodFrequency', 17.8571, ...
    'rcvFilter', 1, 'callMediaFunc', 0), 1, na * P.nSubFrames *
Resource.RcvBuffer(1).numFrames);

rcvI = 1;
for frameI = 1:Resource.RcvBuffer(1).numFrames
    Receive(rcvI).callMediaFunc = 1;
    acqI = 1;
    for subFrameI = 1:P.nSubFrames
        for angleI = 1:na
            Receive(rcvI).framenum = frameI;
            Receive(rcvI).acqNum = acqI;
            acqI = acqI + 1;
            rcvI = rcvI + 1;
        end
    end
end
```

Modify the Event loop to iterate over frames → sub-frames → angles:

```
matlab

for frameI = 1:Resource.RcvBuffer(1).numFrames
    acqI = 1;
    for subFrameI = 1:P.nSubFrames
        for angleI = 1:na
            Event(n).info = 'Full aperture.';
            Event(n).tx = angleI;
            Event(n).rcv = acqI;
            Event(n).recon = 0;
            Event(n).process = 0;
            Event(n).seqControl = 2;
            n = n + 1;
            acqI = acqI + 1;
        end
    end
end
```

Increase the receive buffer rows per frame to accommodate all sub-frames:

```
matlab

Resource.RcvBuffer(1).rowsPerFrame = P.nSubFrames * na * 6144; % adjust as needed
for max depth
```

Add a GUI control to start/stop saving (toggle button) and a callback that enables/disables the external process for the saved events.

```
matlab

    import vsv.seq.uicontrol.VsToggleButtonControl

    UI(3).Control = VsToggleButtonControl('LocationCode','UserB1',...
        'Label','Start Fast Save',...
        'Callback',@saveAcquisition);

    % **** Callback routines used by UI Controls. ****
    function saveAcquisition(~,~,UIValue)
    saveEvents = evalin('base', 'saveEvents');
    Event = evalin('base', 'Event');
    for i = 1:length(saveEvents)
        Event(saveEvents(i)).process = UIValue*2;  %for external
    end
    assignin('base','Event',Event);
    Control = evalin('base','Control');
    if isempty(Control(1).Command), n=1; else, n=length(Control)+1; end
    Control(n).Command = 'update&Run';
    Control(n).Parameters = {'Event'};
    assignin('base','Control', Control);
    end
```

(Optional) Enable acquisition time tags for timing validation:

```
matlab

Resource.VDAS.acqTimeTagEnable = true;
```

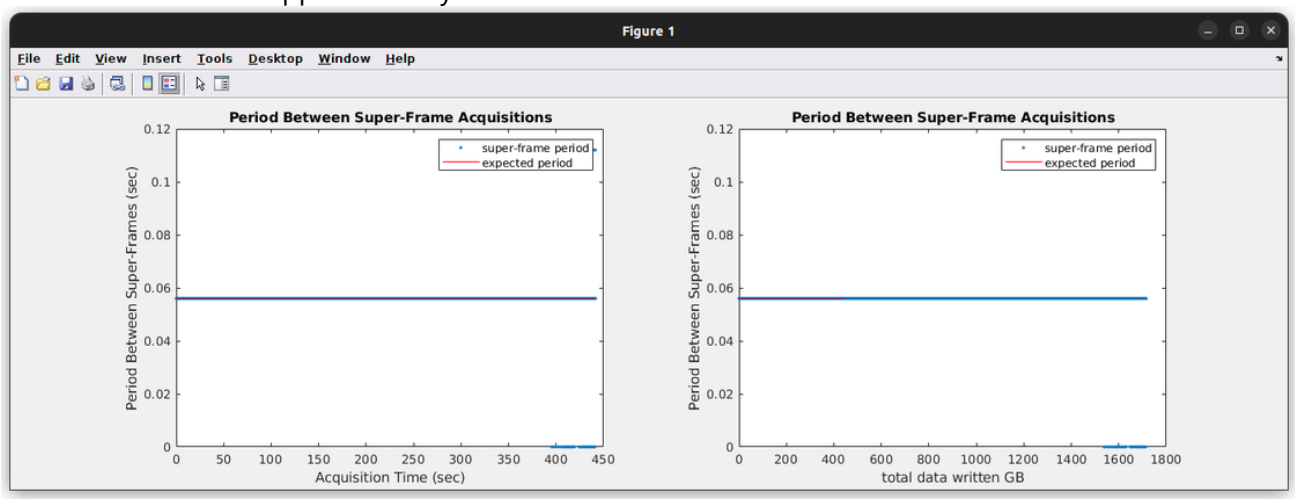
## 6. Test and validate

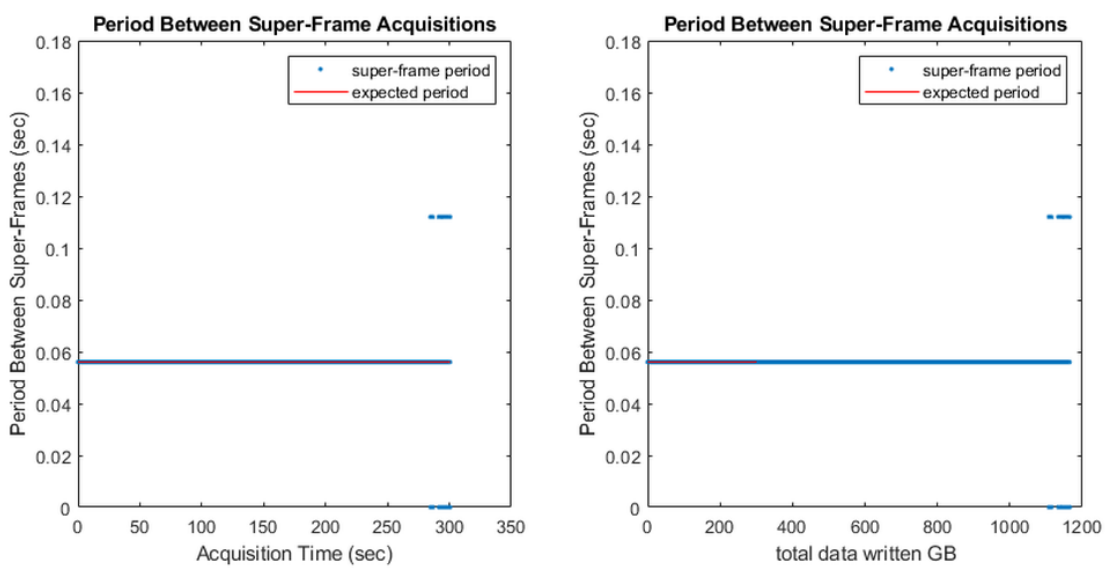
Use or adapt a test runner (`demoRun.m`) to:

1. Verify storage mount points and formatting.
2. Optionally trim/prepare the drives and warn before any destructive wipe.
3. Launch the sequence with profiling enabled.

Once you feel like you have collected enough data:

1. Disable the file streaming with the same GUI button clicked for launching.
2. Run the script: `validateCollectedData`.
3. After a run, validate with `validateCollectedData`.
   The validation plot shows period between frames vs time and vs total storage written; spurious deviations indicate dropped or delayed frames.



## 7. Example summary statistics

Total # files captured: 4530 files

Total duration of acquisition: 254 seconds

Total size of collected data: 0.99 TB

Percent of Storage Used: 25

Average Data Throughput: 3.9 GB/s

With the settings recommended (50 sub-frames per super-frame), the sequence can provide live reconstruction of a 7-angle plane-wave compounding image at ~18 fps while streaming continuously to disk for up to ~17 minutes (assuming the four storage devices sustain ≥ 1 GB/s each after caches fill).

## 8. Recommendations and notes

- Test incrementally: first verify acquisition timing and buffer sizing without FastSave enabled, then enable FastSave and validate transfers.
- Keep the fio benchmark and filesystem tuning guidelines from the main documentation in mind — filesystem choice and mount options materially affect sustained throughput.
- Shut down unnecessary background processes and disable indexing / antivirus on dedicated acquisition drives during tests.
- Record exact parameters (P.nSubFrames, SeqControl arguments, Receive sizes, filesystem, fio job file) alongside the test results for reproducibility.

This document is intended for use by Verasonics' customers only.
Do not duplicate or distribute without permission.